

Concepto previos

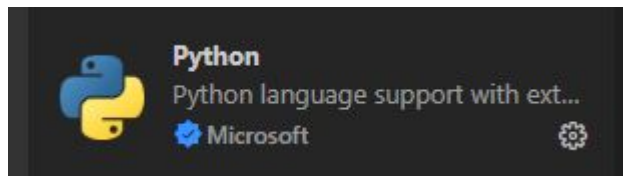
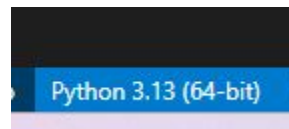
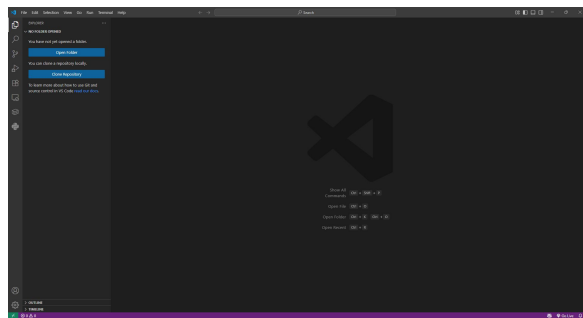
CE.Desarrollo de aplicaciones en lenguaje Python
Curso 2025-2026

INSTALACIÓN PYTHON

- Instalar el intérprete de Python
 - Las diferentes versiones se pueden descargar desde: <https://www.python.org/>

UTILIZAR PYTHON DESDE LA LÍNEA DE COMANDOS

- Editor de código a usar: Visual Studio Code
- Instalar la extensión de **Python de Microsoft**
- Seleccionamos el intérprete: **Ctrl+Shift+P**



- PyCharm
- Editor **Tonny**: Editor para principiantes

TIPOS DE DATOS



Numéricos: Pueden ser

- **Enteros:** tipo int
- **Coma flotante:** tipo float, números decimales
- **Complejos** (complex)

Cadena: “Hola Mundo”

Booleanos: True y False.

OPERADORES. TRABAJANDO CON NÚMEROS

Operador	Símbolo
Suma	+
Resta	-
Multiplicación	*
División	/
División entera	//
Resto de la división	%
Potencia	**
Paréntesis	()
Redondear *	round(a,b)

- El **punto decimal** se indica por medio del símbolo punto (.) y no con la coma (,).

PRÁCTICA

- Elabora un documento donde aparezca el resultado de evaluar las siguientes expresiones:

$7 * 3 - 2 * 4 + 5 * (2 - 3)$

$10 + 2 ** 4$

$45 \% 6$

$18 // 4$

$(12 - 4) * 3 + 2$

$5 ** 2 - 3 * 4$

$100 // (6 + 4)$

$7 * (3 + 2) ** 2$

$50 \% (3 * 5)$

- Comprueba el resultado con la consola

VARIABLES

- **Convenciones de nombres:** Las variables deben tener nombres claros y descriptivos.
- En Python generalmente se suelen escribir en **minúsculas** y **usando guiones** bajos para separar palabras, como ***edad_usuario***. (***Snake case***)
- Python distingue entre mayúsculas y minúsculas (***casesensitive***).
- El signo igual (=) se utiliza dentro del código para asignar un valor a una variable

EJEMPLO

Podemos comprobar si es de un tipo u otro usando la función **isinstance(variable, tipo)**, por ejemplo así:

```
print(isinstance(edad,int))
```

True

```
# Variables en python
```

```
edad = 18 # tipado dinámico  
type(edad)
```

```
nombre = "Luis"  
type(nombre)
```

```
altura = 1.80  
type(altura)
```

```
estudiante = True  
type(estudiante)
```

```
compra = ["Leche", "Pan",  
"Cereal"]  
type(compra)
```


CADENAS DE CARACTERES – FUNCIÓN PRINT()

- **Cadena de caracteres:** Cualquier información que Python intérprete como un texto.
 - 'Hola Mundo'
 - "Hola Mundo"
- **print()** Función que muestra por pantalla la información contenida entre los paréntesis, argumentos.

```
>>> print("Hola qué tal")  
Hola qué tal
```

```
>>> print("Hola","Mundo")  
Hola Mundo
```

CADENAS DE CARACTERES Y PRINT()

- **Concatenar cadenas de caracteres:**
 - El símbolo `+` permite unir cadenas de caracteres.
- No se pueden concatenar cadenas y números con el operador suma.
- En ese caso será necesario convertir los números a cadenas con la función **`str()`**.

CADENAS DE CARACTERES Y PRINT()

- **Repetir** cadenas de caracteres

```
>>> 2*"Hola"  
'HolaHola'
```

- `len()` – **Longitud** cadena

```
>>> len("Hola Mundo")  
10
```

- **Líneas múltiples**

```
"""  
    Esto es una cadena  
    con líneas múltiples  
"""
```

CADENAS DE CARACTERES Y PRINT()

- **Recorrer la cadena: Variante del Bucle for** (Muy importante la indentación)

```
# Recorrer la cadena
for caracter in "Miguel":
    print(caracter)
```

```
cadena = "Miguel"
```

```
for caracter in cadena:
    print(caracter)
```

- **Pertenencia**

```
# Pertenencia
x = "Hola"
print("o" in x) # True
print("o" not in x) # False
```

ÍNDICES EN LAS CADENAS DE TEXTO

- Cada carácter de una cadena de texto queda referenciado por medio de un índice asociado a su posición en la cadena.
- Al primer carácter se le asigna el índice 0, al segundo el 1, al tercero el 2.

```
>>> variable = "melocotón"
>>> variable[0]
'm'
>>> variable[5]
'o'

>>> variable[-1]
'n'
>>> variable[-9]
'm'
```

ÍNDICES EN LAS CADENAS DE TEXTO

- Rebanadas de texto (**slice**)

```
>>> variable = "Melocotón"
>>> variable[0:2]
'Me'
>>> variable[:3]
'Mel'
>>> variable[3:]
'ocotón'
```

ÍNDICES EN LAS CADENAS DE TEXTO

- F-strings o **Cadenas Formateadas**
- f'...' o f"...": Introducidas en Python 3.6, permiten incluir expresiones y variables dentro de una cadena usando {}.

Cadena preformateadas

```
nombre = "Miguel"  
edad = 24
```

```
print(f"Mi nombre es {nombre} y tengo {edad} años")
```

```
valor1 = 1.45  
valor2 = 45.45  
print(f'{valor1:6.3f}')  
print(f'{valor2:6.3f}')
```

```
print("{} tiene {} años".format(nombre,edad))  
print("%s tiene %d años"%(nombre,edad)) # %s indica que nombre es cadena y %d que  
edad es entero
```

PASANDO VALORES AL PROGRAMA -INPUT()

- Función que permite introducir información a través del teclado
 - `nombreVariable = input("mensaje")`
- Por defecto `input()` convierte la respuesta en una cadena de caracteres, así si intentamos hacer un cálculo obtendremos un mensaje de error.
- Esta función sólo admite un único argumento

Pasando valores al programa – input()

```
nombre = input("Introduce tu nombre: ")  
edad = int(input("Introduce tu edad: "))  
peso = float(input("Introduce tu peso: "))
```

```
print(f" Tu nombre es {nombre}, tienes {edad} y pesas {peso}")
```


PRIMER PROGRAMA: HOLA MUNDO

- Creamos el archivo holamundo.py y añadimos:

```
print("Hola Mundo")
```

- Comentarios

```
# Ejemplo de comentario
```

```
"""
```

```
Comentario multilínea
```

```
"""
```

PRIMEROS PROGRAMAS

1. Realiza un programa cuya salida sea similar a la que se muestra en la imagen siguiente.

```
Programa para realizar Cálculos
Escribe el primer número: 23
Escribe el segundo número: 6
Operaciones con 23.0 y 6.0
23.0 + 6.0 = 29.0
23.0 - 6.0 = 17.0
23.0 * 6.0 = 138.0
23.0 / 6.0 = 3.83
23.0 // 6.0 = 3.0
23.0 % 6.0 = 5.0
```

PRIMEROS PROGRAMAS

2. Realiza un programa cuya salida sea similar a la que se muestra en la imagen siguiente.

```
Programa que analiza una palabra
Escribe una palabra.: Informática
La palabra Informática tiene 11 caracteres
La primera letra es I
La última letra es a
Las dos primeras letras son In
Las dos últimas letras son ca
```

SENTENCIAS

CONDICIONALES: IF...

ELIF...ELSE...

SENTENCIA CONDICIONAL IF... (UNA OPCIÓN)

- Sentencia condicional if... (Una opción)

if condición:

bloque de órdenes que se ejecutarán si la condición es cierta.

Estas órdenes pueden ocupar más de una línea.

Condición	es una expresión lógica que el programa evaluará y que generará un valor cierto o falso.
:	Símbolo que marca el final de la expresión lógica. Es obligatoria.
Bloque de órdenes	Conjunto de órdenes que se ejecutan solamente si la condición lógica ha sido evaluada como verdadera.

- Todas las órdenes asociadas al if han de tener el mismo sangrado.

COMPARADORES RELACIONALES

Símbolo	Comparador	Genera True	Genera False
>	Mayor que	$3 > 2$	$2 > 3$
<	Menor que	$2 < 3$	$3 < 2$
>=	Mayor o igual que	$2 \geq 1+1$	$2 \geq 1+2$
<=	Menor o igual que	$4-2 \leq 1$	$4-2 \leq 2$
==	Igual que	$2 == 1+1$	$2 == 1$
!=	Distinto de	$6/2 != 2$	$6/2 != 3$

OPERADORES LÓGICOS

Símbolo	Operador	Genera True	Genera False
and	Y lógico	true and true	true and false false and false false and true
or	O lógico	true or true true or false false or true	false or false
not	Negación	not false	not true

IF ... ELSE ... (DOS OPCIONES)

if condición:

 bloque de órdenes que se ejecutarán si la condición es cierta.

else:

 Bloque de órdenes que se ejecutarán si la condición es falsa.

Todas las órdenes de un bloque han de tener el mismo sangrado, cada bloque puede tener un sangrado distinto... aunque eso reduce la claridad del código.

EJERCICIO 3

- Diseña un programa que pregunte la edad al usuario y la almacene en la variable 'edad'.
- A continuación comprobad si la edad es inferior a 18 años.
- Si la condición es cierta el programa debe avisar por pantalla que el usuario es menor de edad y si es falsa de que es mayor de edad.

IF... ELIF... ELSE... (MÁS DE DOS OPCIONES)

if condición1:

 bloque de órdenes que se ejecutarán si la condición1 es cierta.

elif condición2:

 bloque de órdenes que se ejecutará si la condición2 es cierta.

else:

 bloque de órdenes que se ejecutarán si todas las condiciones son falsas.

- Se pueden incluir tantos bloques elif como sean necesarios

Observa que en este tipo de sentencia las opciones son siempre mutuamente excluyentes. En cualquier caso solamente se ejecutará uno de los bloques.

EJERCICIO 4

- Diseña un programa que indique si un número:
 - es múltiplo de dos,
 - de dos y de cuatro o
 - si no es múltiplo ni de dos ni de cuatro.